

FLEX REWARD SIMULATION

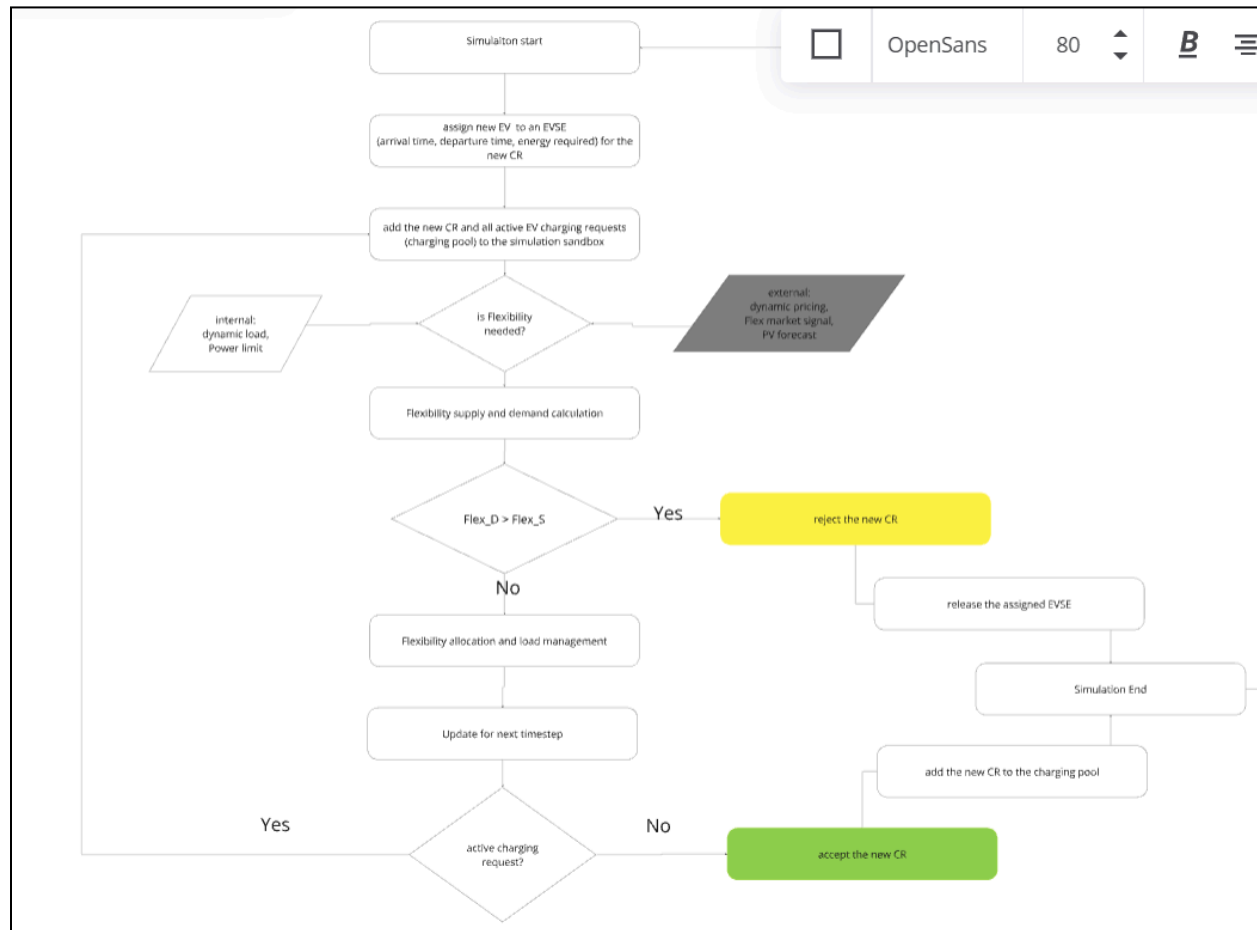
Introduction

The FlexRewardSimulation document presents a comprehensive overview of a simulated environment for electric vehicle (EV) smart charging, developed to optimize charging requests within constraints such as power availability and vehicle requirements. It introduces essential classes, including:

- CarSpecs for detailing EV attributes,
- ChargingPoint for managing EV charging stations,
- AvailableFlexibilityRequest for tracking EV charging needs and timeframes,
- FlexibilityCalculator to compute flexibility in charging power and time, and
- FlexibilitySimulation for allocating power based on real-time demands and flexibility.

Supporting utility classes, like PlottingUtils, generate visualizations for power distribution and flexibility contributions across charging points, aiding in insights for load management.

Simulation Process



This flowchart illustrates the decision-making process in an EV charging simulation, focusing on how flexibility is managed to accommodate new charging requests (CRs). Here's a breakdown of the key steps:

1. **Simulation Start:** The simulation begins, initializing with a new EV charging request.
2. **Assigning New EV to EVSE:** The system assigns an EV to an Electric Vehicle Supply Equipment (EVSE) with details such as arrival time, departure time, and energy requirement.
3. **Adding Requests to Simulation:** The new CR and all active charging requests are added to the charging pool in the simulation environment.
4. **Determining Flexibility Needs:**
 - The system checks if flexibility is required based on **internal** factors (like dynamic load and power limits) and **external** factors (such as dynamic pricing, flexibility market signals, and PV forecasts).

5. **Flexibility Supply and Demand Calculation:** The system calculates flexibility demand (Flex_D) and supply (Flex_S) to assess if there is enough capacity to fulfill the new request.
6. **Decision Based on Flexibility:**
 - If **Flex_D > Flex_S**: The new CR is rejected, and the assigned EVSE is released.
 - If **Flex_D ≤ Flex_S**: The system allocates the necessary flexibility and manages the load.
7. **Update and Check for Active Charging Requests:**
 - After allocation, the system updates for the next timestep.
 - It checks if there are still active charging requests.
 - If **yes**: The process loops to manage further requests.
 - If **no**: The new CR is accepted, and the EVSE is added to the charging pool.
8. **Simulation End:** Once all conditions are met, the simulation concludes.

System/Code Overview:

CarSpecs Class:

The **CarSpecs** class is designed to represent the core specifications of an electric vehicle (EV) for use in a smart charging simulation. This class encapsulates essential details of the EV, such as its make, model, manufacturing year, battery capacity, and state of charge (SOC).

Attributes:

1. **make (str)**
2. **model (str)**
3. **year (int)**
4. **battery_capacity_in_kwh (float)**
This attribute defines the car's total battery capacity in kilowatt-hours (kWh).
5. **initial_soc (float)**
The **initial_soc** attribute represents the initial state of charge of the car's battery, set as a percentage of the total capacity (ranging from 0.0 to 100.0).
6. **soc (float)**
This attribute stores the current state of charge of the battery, also as a percentage. Unlike **initial_soc**, **soc** can change during the simulation as charging or discharging occurs. This provides real-time feedback on the battery status, allowing for control and adjustments as the simulation progresses.

ChargingPoint Class:

The `ChargingPoint` class represents an individual EV charging station within a smart charging simulation, equipped with basic attributes and methods to manage its availability, track the ongoing charging session, and handle nominal power capacity.

Attributes:

1. **evse_id (int or str)**
This attribute uniquely identifies the charging point within the system.
2. **available (bool)**
A boolean flag indicating the availability of the charging point. If `True`, the charging point is open and ready to accept a new charging request. If `False`, it is currently in use and unavailable for new requests. This attribute can be updated as charging sessions begin and end.
3. **current_request**
This attribute stores the current charging request assigned to the charging point. When a charging session is active, `current_request` holds a reference to the EV charging request; otherwise, it is set to `None`. This allows the simulation to manage and track ongoing charging sessions effectively.
4. **nominal_power_cp (float)**
The nominal power output capacity of the charging point, expressed in kilowatts hours (kWh).

Methods:

1. **assign_request(request)**
Assigns a new charging request to the charging point by setting `current_request` to the provided `request`.
2. **finish_charging()**
Ends the current charging session by setting `current_request` to `None`, effectively clearing the request. This method can be called when a charging session completes, making the charging point ready for new requests.
3. **setAvailable(val)**
Updates the `available` attribute to the specified boolean value, `val`. e.g . `True` or `False`.
4. **isAvailable()**
Returns the current availability status of the charging point by checking the `available` attribute.

AvailableFlexibilityRequest Class:

The `AvailableFlexibilityRequest` class represents a request for flexibility in the context of an EV charging simulation. This class stores details of a charging session, including the EV's battery specifications, requested energy, and timing details, to facilitate smart scheduling and flexibility in load management. It includes properties and methods to validate the request, calculate the target state of charge (SoC), and manage session updates, making it suitable for advanced simulations that factor in energy needs and time constraints for each EV charging session.

Attributes:

1. **session_id (str)**
evse_id (ChargingPoint)
An instance of `ChargingPoint`, representing the specific charging station where the request is assigned.
2. **requested_energy (float)**
The amount of energy (in kW) requested for this charging session.
3. **requested_leave_time (datetime)**
The desired departure time for the EV, represented as a `datetime` object. This value is calculated based on the `arrival_time` and requested charging duration and is validated to ensure it is later than both the current time and the arrival time.
4. **arrival_time (datetime)**
The arrival time of the EV at the charging station, stored as a `datetime` object.
5. **car_specs (CarSpecs)**
An instance of `CarSpecs`, storing the specifications of the EV associated with this request. It includes the EV's battery capacity, initial SoC, and other relevant attributes for charging simulation.
6. **charged_energy (float)**
The amount of energy (in kW) that has been charged so far in this session. This value starts at 0 and increases as the charging session progresses.
7. **charged_time (float)**
The amount of time (in hours) spent charging in this session, useful for monitoring and tracking session duration.
8. **charge_complete (bool)**
A boolean indicating whether the charging session is complete.
9. **target_soc (float)**
The target state of charge for the EV's battery, calculated based on the requested energy and battery capacity. It is capped at 100% to prevent overcharging.
10. **time_flexibility (float)**
This attribute represents the flexibility in time for this charging session. It starts at 0 but can be adjusted to reflect allowable variations in charging time.
11. **power_flexibility (float)**
The flexibility in power output for the session. This attribute allows for dynamic adjustment to manage power load within the charging network.

12. **flexibility_contribution (float)**

A measure of the flexibility contribution of this session, used to adjust power or time requirements within the simulation.

13. **flexibility_contribution_per_timestep (list)**

A list recording the flexibility contributions at each timestep.

PlottingUtils Class:

The `PlottingUtils` class is a utility class that provides static methods to generate visualizations for EV charging simulations. It includes functions to create bar charts that illustrate power supplied to each EV and the flexibility contributions (reductions in energy consumption) at various time steps.

Methods:

1. **plot_power_supplied(completed_requests)**

- Generates a bar chart that visualizes the power supplied to each EV at each time step, starting from their respective arrival times. This plot allows users to track how much power each EV received at specific intervals, aiding in understanding power distribution over time.

2. **plot_flexibility_contribution(completed_requests)**

- Generates a bar chart that visualizes each EV's flexibility contributions at each time step, starting from their respective arrival times.

Flexibility Utils

Overview:

This module provides utility functions to simulate random EV charging requests. It generates instances of `CarSpecs` and `AvailableFlexibilityRequest` for various vehicles, assigning them to specific charging points.

Functions:

1. **generate_random_car()**

- **Description:**

Creates a random `CarSpecs` instance based on predefined car models, with random values for battery state of charge (SoC) within a specified range. This

function simulates different EVs arriving at a charging station with varying initial conditions.

2. `generate_random_request(id, connector, car)`

- **Description:**

Generates an `AvailableFlexibilityRequest` for a given EV and charging point, including random values for arrival time, requested energy, and requested leave time. This function simulates each EV's individual charging needs and time constraints.

3. `generate_random_requests(num_requests, connectors)`

- **Description:**

Creates a list of random charging requests by generating random cars and associating them with available connectors. Charging points are assigned in a round-robin pattern to balance the load.

FlexibilityCalculator Class

Overview:

The `FlexibilityCalculator` class provides methods to calculate the flexibility of an EV charging request based on time and power. Flexibility represents the ability to adjust the charging session based on the EV's constraints and the charging point's capacity. It is crucial for optimizing charging schedules and accommodating changes in energy demand without exceeding available power resources.

Functions

1. `calculate_time_flexibility(charging_request, nominal_power_cp)`

- **Description:**

Calculates the "time flexibility" for a given charging request. Time flexibility is the difference between the maximum allowable time left before the EV must be fully charged and the minimum time required to complete the charge. This metric is vital for understanding how much delay could be applied to the charging process without violating the user's leave time.

- **Parameters:**

- `charging_request` (`AvailableFlexibilityRequest`): The charging request, containing details such as requested energy, arrival time, and required leave time.
- `nominal_power_cp` (float): The nominal power capacity (in kW) of the charging point, indicating the maximum power output it can deliver.

- **Returns:**
 - **float:** The calculated time flexibility in minutes.
- **Logic:**
 - **Charged Time:**

The time the EV has already been charging (**charged_time**) is obtained from the **charging_request**, which is in minutes.
 - **Remaining Charging Time:**

This is calculated as the time left between the requested leave time and the arrival time, minus the already-charged time. It represents the maximum time remaining that the car can stay connected to the charging point.
 - **Required Charging Time:**

Using the requested energy that is still needed (**requested_energy - charged_energy**), we calculate the time needed at full power (**nominal_power_cp**) to complete the charging.
 - **Time Flexibility Calculation:**

The time flexibility is the difference between the remaining charging time and the required charging time. A positive value indicates the amount of time the charging process could be delayed without impacting the user's requested leave time.

2. **calculate_power_flexibility(charging_request, nominal_power_cp)**

- **Description:**

Calculates the "**power flexibility**" for a given charging request. Power flexibility refers to how much the charging power can be adjusted downwards based on available time flexibility. If time flexibility allows, charging power could be reduced to delay completion, which aids in managing the total load across multiple charging points.
- **Parameters:**
 - **charging_request** (AvailableFlexibilityRequest): The charging request with details on requested energy and leave time.
 - **nominal_power_cp** (float): The nominal charging power (in kW) of the charging point.
- **Returns:**
 - **float:** The calculated power flexibility in kW.
- **Logic:**
 - **Determine Time Flexibility:**

Calls **calculate_time_flexibility** to check if there is sufficient time flexibility (greater than or equal to 15 minutes).
 - **Calculate Power Flexibility:**

- **If time flexibility is sufficient (≥ 15 minutes):**

The method returns the full nominal power of the charging point, meaning the full charging rate can be adjusted.

```
a. power_flexibility = nominal_power_cp
```

- **If time flexibility is insufficient (< 15 minutes):**

Power flexibility is scaled based on the ratio of available time flexibility to 15 minutes, providing partial power flexibility proportionate to the remaining time.

```
a. power_flexibility = (time_flexibility / 15) *  
    nominal_power_cp
```

FlexibilitySimulation Class

The `FlexibilitySimulation` class manages multiple EV charging requests by:

- Accepting and tracking requests based on arrival time.
- Calculating total flexibility in power and time for each request.
- Allocating power according to availability and flexibility.
- Rejecting requests if flexibility is insufficient to meet demand.

Class Attributes

1. **power_supply**: The total power (in kW) available for charging all EVs in the simulation.
2. **time_step**: The time increment (in minutes) for updating the simulation time.
3. **queued_requests**: List of charging requests currently being processed.
4. **pending_requests**: Requests that have not yet arrived but will be added once their arrival time comes.
5. **completed_requests**: Requests that have been successfully charged to their requested energy.
6. **rejected_requests**: Requests that were rejected due to insufficient power or flexibility.
7. **current_time** and **start_time**: Track the current and starting time for the simulation.

Functions

1. flexibility_demand

- Calculates the total power demand based on `queued_requests`.
- For each request, check if it needs more energy based on remaining time and power required.
- Computes the flexibility demand as the difference between total demand and available power.
- **Returns:** The amount of additional power required beyond the current supply, ensuring a minimum of `0.0`.

$$\text{Flexibility demand} = \sum C P_{\text{rated power}} - \text{total power supply}$$

2. flexibility_supply

- Calculates the total flexibility supply by summing the power flexibility across all queued requests.
- Calls `calculate_power_flexibility` from `FlexibilityCalculator` to determine flexibility for each request.
- **Returns:** Total power flexibility available.

$$\text{POWER FLEXIBILITY} = \sum \text{Power_Flexibility (of all active charging)}$$

3. add_request and reject_new_request

- `add_request` adds a new request to the queue, marking it as accepted.
- `reject_new_request` removes the latest request due to insufficient flexibility and logs the rejection.

4. allocate_power

- Allocates power to each EV, updating its charging status based on available power.
- Calculates the potential energy that could be charged in the given time step, ensuring it doesn't exceed the requested energy.
- Moves the request to `completed_requests` once fully charged.

5. allocate_flexibility_and_load_management

- Manages power allocation across requests, handling situations where demand exceeds supply.
- **Steps:**
 - **Demand Check:** If instantaneous demand is below supply, each request receives full nominal power.
 - **Allocation Factor:** If demand exceeds supply, an allocation factor is applied to adjust power flexibly across requests.
- **Allocation Factor Calculation:**

- **Purpose:** To fairly distribute limited power across requests.
- **Formula:**
$$\text{allocation_factor} = \frac{\{\text{total_power_demand}\} - \{\text{power_supply}\}}{\{\text{flexibility_supply}\}}$$
- The `allocation_factor` scales the amount of power each request receives based on its flexibility. A higher allocation factor results in reduced power for flexible requests.

6. `update_for_next_timestep`

- Advances the simulation by `time_step` minutes, updating `current_time` and `charged_time` for each request.

7. `run_simulation`

- Manages the overall simulation loop:
 - Handles new requests as they arrive.
 - Checks if flexibility demand exceeds flexibility supply and applies load management.
 - Moves the simulation to the next time step.

Example

```
if __name__ == '__main__':

    num_connectors = 5

    connectors = [ChargingPoint(i + 1) for i in range(num_connectors)]

    connectors_in_use = []

    power_supply = 40.0

    time_step = 15

    car1 = CarSpecs(make="Tesla", model="Model S", year=2022,
battery_capacity_in_kwh=100, initial_soc=50)

    request1 = AvailableFlexibilityRequest("user1", connectors[0],
11.0,90, datetime.now(), car1, 0, 0)

    car2 = CarSpecs(make="Tesla", model="Model X", year=2022,
battery_capacity_in_kwh=100, initial_soc=50)
```

```
request2 = AvailableFlexibilityRequest("user2", connectors[1], 11.0,
60, datetime.now(), car2, 0, 0)

car3 = CarSpecs(make="Tesla", model="Model Z", year=2022,
battery_capacity_in_kwh=100, initial_soc=50)

request3 = AvailableFlexibilityRequest("user3", connectors[2], 11.0,
60, datetime.now(), car3, 0, 0)

car4 = CarSpecs(make="Tesla", model="Model A", year=2022,
battery_capacity_in_kwh=100, initial_soc=50)

request4 = AvailableFlexibilityRequest("user4", connectors[3], 11.0,
60, datetime.now(), car4, 0, 0)

simulation = FlexibilitySimulation(power_supply, time_step)

requests = [request1, request2, request3, request4]

# create a copy of the requests

copy_requests = copy.deepcopy(requests)

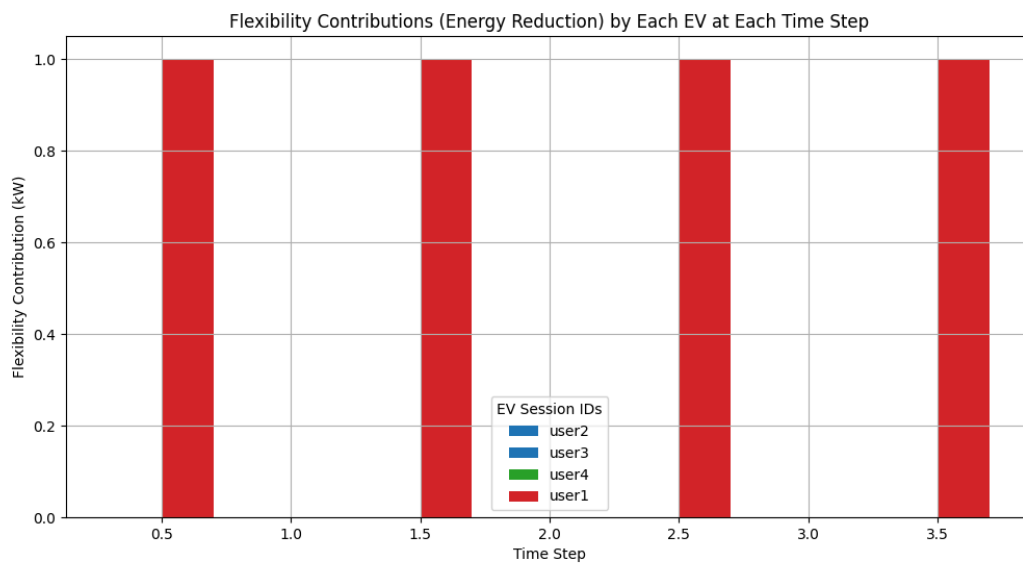
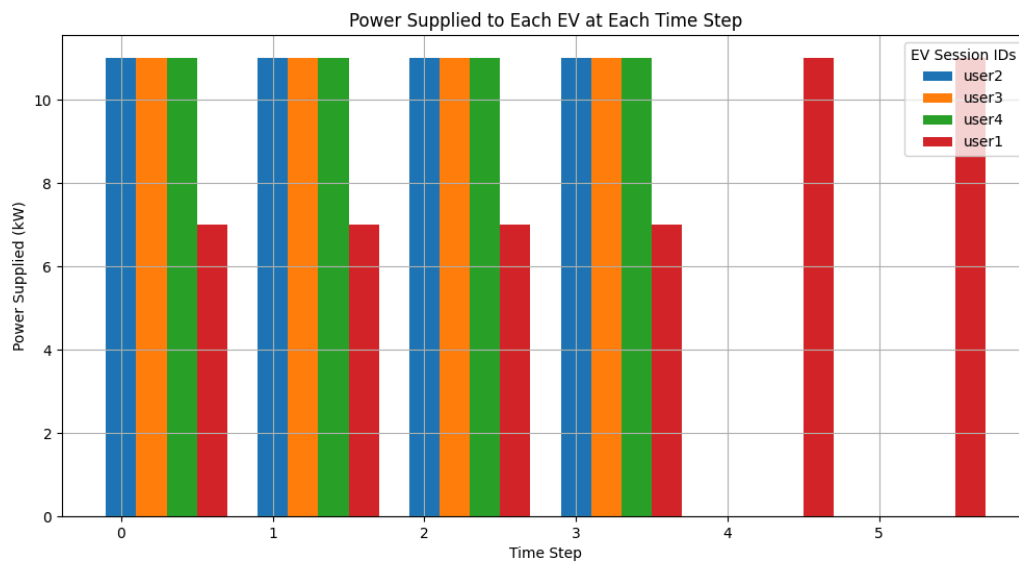
simulation.run_simulation(copy_requests)

PlottingUtils.plot_power_supplied(simulation.completed_requests)

PlottingUtils.plot_flexibility_contribution(simulation.completed_requests)
```

This code simulates the charging requests of four vehicles at a set of charging points. It sets up the parameters for the simulation, creates requests based on current vehicle specifications, and runs the simulation to allocate power based on user demands and available supply. Finally, it visualizes the results to provide insights into how the power was distributed among users during the simulation period.

Visualizations can be seen below:



Logs

```
New request added to the queue: user1 at 2024-10-31 00:36:23
Request user1 has been accepted.
New request added to the queue: user2 at 2024-10-31 00:36:23
Request user2 has been accepted.
New request added to the queue: user3 at 2024-10-31 00:36:23
Request user3 has been accepted.
New request added to the queue: user4 at 2024-10-31 00:36:23
Request user4 has been accepted.
Flexibility Demand 4.0
Flexibility Supply 11.0
Demand exceeds supply, applying flexibility.
Allocated 7.00 kW to user1
Allocated 11.00 kW to user2
Allocated 11.00 kW to user3
Allocated 11.00 kW to user4
Time: 2024-10-31 00:51:23
Current Time Step : 1.0
-----
Flexibility Demand 4.0
Flexibility Supply 11.0
Demand exceeds supply, applying flexibility.
Allocated 7.00 kW to user1
Allocated 11.00 kW to user2
Allocated 11.00 kW to user3
Allocated 11.00 kW to user4
Time: 2024-10-31 01:06:23
Current Time Step : 2.0
-----
Flexibility Demand 4.0
Flexibility Supply 10.999999999999995
Demand exceeds supply, applying flexibility.
Allocated 7.00 kW to user1
Allocated 11.00 kW to user2
Allocated 11.00 kW to user3
Allocated 11.00 kW to user4
Time: 2024-10-31 01:21:23
Current Time Step : 3.0
-----
Flexibility Demand 4.0
```

Flexibility Supply 10.000000000000005

Demand exceeds supply, applying flexibility.

Allocated 7.00 kW to user1

Allocated 11.00 kW to user2

user2 is fully charged with 11.00 kWh.

Allocated 11.00 kW to user3

user3 is fully charged with 11.00 kWh.

Allocated 11.00 kW to user4

user4 is fully charged with 11.00 kWh.

Time: 2024-10-31 01:36:23

Current Time Step : 4.0

Allocated 11.00 kW to user1

Time: 2024-10-31 01:51:23

Current Time Step : 5.0

Allocated 11.00 kW to user1

user1 is fully charged with 11.00 kWh. At time 2024-10-31 01:58:12

Time: 2024-10-31 02:06:23

Current Time Step : 6.0

Flexibility contribution of user2: 0.00 kW

Flexibility contribution of user3: 0.00 kW

Flexibility contribution of user4: 0.00 kW

Flexibility contribution of user1: 4.00 kW